DRC / LVS VERIFICATION DASHBOARD

Web-Deployed IC Physical Verification System

Project Title	DRC/LVS Verification Dashboard
Developer	Lokpavan P
Technology	mock_28nm-v3.1
Tool Version	mock_icv-2024.12
Framework	Python 3.11 + Streamlit
Report Date	February 27, 2026
Version	v1.0.0

Developed by Lokpavan P

Submitted as part of EDA Automation & Software Engineering Portfolio

Lokpavan P

1. Executive Summary

This report documents the complete design, development, and deployment of the DRC/LVS Verification Dashboard — a web-based IC physical verification tool built by Lokpavan P. The system enables semiconductor design engineers to upload layout files, execute Design Rule Check (DRC) and Layout vs. Schematic (LVS) verification flows, and receive sign-off reports directly through a browser-based interface.

The project is architected to mirror industry-standard EDA flows used in Synopsys Custom Compiler, Synopsys IC Validator (ICV), and Cadence Virtuoso/Pegasus environments. It demonstrates real-world software engineering skills including Python automation, subprocess orchestration, TCL scripting, Streamlit UI development, and cloud deployment — all directly aligned with the Synopsys Software Engineering Apprenticeship job requirements.

Metric	Value
Total Source Files	8 files (Python × 4, TCL × 1, Streamlit × 1, Config × 2)
DRC Rules Implemented	10 rules (mock_28nm PDK — Metal, Poly, Diffusion, Via, N-Well layers)
LVS Error Categories	8 categories (NET_OPEN, SHORT, DEVICE_MISMATCH, FLOATING, etc.)
Report Outputs	summary_report.txt, drc_violations.csv, lvs_violations.csv
Deployment Targets	Local (Windows/Linux), Streamlit Cloud, Render.com
Engine Sign-off	PASS / FAIL with machine-parseable [SIGNOFF] token

Lokpavan P

2. Project Objectives

The primary objectives of the DRC/LVS Verification Dashboard are:

- Build a fully deployable, browser-based EDA verification interface requiring zero GUI tool installation.
- Implement a realistic DRC rule deck using mock_28nm PDK parameters for Metal, Poly, Diffusion, Via, and N-Well layers.
- Implement an LVS netlist comparator covering device mismatches, net opens, shorts, floating nodes, and port mismatches.
- Produce EDA-style sign-off reports (TXT summary + CSV violation tables) matching Synopsys ICV and Calibre output formats.
- Demonstrate clean software engineering architecture: strict separation of UI layer, business logic, and report generation.
- Enable one-command cloud deployment suitable for team-wide or portfolio demonstration.

Lokpavan P

3. System Architecture

3.1 Architectural Overview

The system follows a strict three-layer architecture that mirrors how production EDA automation systems are structured in industry:

Layer	Component	Responsibility
Presentation	app.py (Streamlit)	File upload, job submission, result display, report download
Orchestration	python/main.py	CLI entry point, subprocess coordination, sign-off emission
Verification	drc_engine.py	DRC rule deck execution, violation detection, severity tagging
Verification	lvs_engine.py	Netlist extraction, schematic comparison, LVS error classification
Reporting	report_generator.py	EDA-style TXT + CSV report generation
TCL Flow	tcl/mock_drc_lvs.tcl	Simulates tclsh-based ICV batch flow with phase logging

3.2 Data Flow

The end-to-end data flow through the system is:

- User uploads a Layout JSON file via the Streamlit browser UI.
- app.py writes the file to a secure temporary directory and invokes python/main.py via subprocess.
- main.py parses the JSON, calls run_drc() and run_lvs() from the engine modules, and collects violation objects.
- report_generator.py serialises violations into summary_report.txt, drc_violations.csv, and lvs_violations.csv.
- main.py emits the machine-parseable [SIGNOFF] STATUS=PASS|FAIL line to stdout.
- app.py captures stdout, parses the sign-off token, loads CSVs with pandas, and renders the dashboard.
- Download buttons serve the generated report files directly from the temp directory.

Lokpav

4. Project Folder Structure

```
drc_lvs_dashboard/ │ ├── app.py                    ← Streamlit UI layer │ ├── python/
│   ├── main.py               ← CLI entry point (invoked via subprocess) │   ├──
drc_engine.py          ← DRC rule-deck executor │   ├── lvs_engine.py          ← LVS
netlist comparator │   └── report_generator.py    ← TXT + CSV report builder │ ├──
tcl/ │   └── mock_drc_lvs.tcl       ← TCL mock flow (tclsh) │ ├── .streamlit/ │   └──
config.toml            ← Dark theme + server config │ ├── sample_layout.json         ←
Pre-built test case (INVERTER_28NM) ├── requirements.txt           ← streamlit, pandas
└── README.md                 ← Full setup + deployment guide
```

Lokpavan P

5. Module Descriptions

5.1 app.py — Streamlit UI Layer

The Streamlit application serves as the sole user-facing layer. It is responsible for file ingestion, subprocess orchestration, result rendering, and report download. It contains no DRC or LVS logic — all verification is delegated to the engine via subprocess. Key capabilities include:

- Drag-and-drop layout JSON upload with file validation and size display.
- Single-click 'Run DRC/LVS' button that triggers both TCL and Python phases.
- Metric cards displaying design name, technology, DRC violations, LVS violations, and runtime.
- Colour-coded PASS (green) / FAIL (red) sign-off banner.
- Interactive violation tables with severity-based row colouring via pandas Styler.
- One-click download buttons for all three report files.
- Collapsible engine log viewer showing raw Python + TCL output.

5.2 python/main.py — CLI Orchestrator

The main.py module is the CLI entry point invoked by the Streamlit layer via subprocess. It mirrors the invocation pattern of Synopsys ICV batch runs (icv -runset rule.drc -layout cell.gds). Responsibilities:

- Argument parsing via argparse: --layout and --outdir flags.
- JSON layout loading with schema validation and error handling.
- Sequential invocation of run_drc() and run_lvs() engine functions.
- Delegation of report generation to report_generator.py.
- Emission of the machine-parseable [SIGNOFF] STATUS=PASS|FAIL DRC_VIOLS=N LVS_VIOLS=N line.
- Exit code 0 (PASS) or 1 (FAIL) for shell-level sign-off integration.

5.3 python/drc_engine.py — DRC Rule Deck

Implements a 10-rule mock_28nm DRC deck covering the most critical physical verification checks. Rule naming follows Synopsys SVRF/ELT conventions.

Rule	Description	Min Value	Layer
M1.W.1	Metal-1 minimum width	0.09 µm	Metal-1
M1.S.1	Metal-1 minimum spacing	0.10 µm	Metal-1
V1.EN.1	Via-1 enclosure by Metal-1	0.04 µm	Via-1
POLY.W	Poly minimum width	0.03 µm	Poly

Lokpavan P

DIFF.S	Diffusion minimum spacing	0.12 µm	Diffusion
NWELL.W	N-well minimum width	0.30 µm	N-Well
M2.W.1	Metal-2 minimum width	0.10 µm	Metal-2
M2.S.1	Metal-2 minimum spacing	0.10 µm	Metal-2
CONT.EN	Contact enclosure by diffusion	0.04 µm	Contact
GC.OV	Gate-cut overlap on poly	0.02 µm	Gate

5.4 python/lvs_engine.py — LVS Comparator

Implements LVS netlist comparison covering 8 error categories aligned with Calibre LVS and Synopsys StarRC output taxonomy. Each violation includes: error type, description, net/device name, schematic value, layout value, and severity.

Error Type	Description
NET_OPEN	Net open — missing connection between nodes
NET_SHORT	Net short — unintended connection detected
DEVICE_MISMATCH	Device mismatch — W/L ratio does not match schematic
MISSING_DEVICE	Missing device — element in schematic absent from layout
EXTRA_DEVICE	Extra device — layout element has no schematic counterpart
PORT_MISMATCH	Port mismatch — pin name or direction inconsistency
PARAM_MISMATCH	Parameter mismatch — device parameter outside tolerance
FLOATING_NODE	Floating node — net with no drive or load

5.5 tcl/mock_drc_lvs.tcl — TCL Mock Engine

A TCL script invoked via tclsh that simulates the phased logging of a Synopsys IC Validator or Cadence Pegasus batch flow. It emits [INFO] log lines for DRC phase (rule deck loading, flattenning, layer derivations, rule checks) and LVS phase (netlist extraction, device/net comparison). The Python orchestrator captures this output and presents it in the engine log viewer. The TCL engine is gracefully skipped if tclsh is unavailable, maintaining full functionality in cloud environments.

6. Layout JSON Input Schema

The system accepts layout data as a structured JSON file. This format represents a simplified OpenAccess / GDSII-derived layout description. The schema is:

Field	Type	Description
design_name	string	Cell/design identifier (e.g. INVERTER_28NM)
technology	string	PDK identifier (e.g. mock_28nm)
top_cell	string	Top-level cell name
layers[].name	string	Layer name (M1, POLY, DIFF, M2, etc.)
layers[].geometries[].x	float	X coordinate of geometry origin (µm)
layers[].geometries[].y	float	Y coordinate of geometry origin (µm)
layers[].geometries[].width	float	Geometry width — checked against min-width rules (µm)
layers[].geometries[].spacing	float	Spacing to adjacent shape — checked against spacing rules (µm)
nets[].name	string	Net identifier (VDD, VSS, CLK, etc.)
nets[].connections	integer	Number of terminals connected to this net
devices[].name	string	Device instance name (MN0, MP0, etc.)
devices[].type	string	Device type (NMOS, PMOS, RES, CAP, etc.)
devices[].schematic_width	float	Reference width from schematic netlist (µm)
devices[].layout_width	float	Extracted width from layout (µm)

Lokp

7. Verification Results — Sample Run

The following results were obtained by running the engine against the included sample_layout.json file (INVERTER_28NM cell):

Parameter	Result
Design Name	INVERTER_28NM
Technology	mock_28nm
DRC Total Violations	8 (7 Errors, 1 Warning)
LVS Total Violations	3 (3 Errors, 0 Warnings)
Overall Sign-off Status	FAIL ✗
Runtime	< 1 second

7.1 DRC Violations Observed

- M1.W.1 ERROR — Metal-1 width 0.07 µm violates 0.09 µm minimum at (0.30, 0.22)
- M1.S.1 ERROR — Metal-1 spacing 0.08 µm violates 0.10 µm minimum at (0.43, 0.50)
- POLY.W ERROR — Poly width 0.025 µm violates 0.03 µm minimum at (0.43, 0.19)
- DIFF.S ERROR — Diffusion spacing 0.10 µm violates 0.12 µm minimum (×2 instances)
- M2.W.1 ERROR — Metal-2 width 0.08 µm violates 0.10 µm minimum
- M2.S.1 ERROR — Metal-2 spacing 0.09 µm violates 0.10 µm minimum
- M2.W.1 WARNING — Metal-2 near-limit at 0.101 µm (rule: 0.10 µm)

7.2 LVS Violations Observed

- NET_OPEN ERROR — Net 'OUT' has only 1 terminal (schematic requires ≥2 connections)
- DEVICE_MISMATCH ERROR — Device MN0: layout W=0.30 µm vs. schematic W=0.28 µm (7.1% deviation)
- FLOATING_NODE ERROR — Net 'OUT' flagged as floating (no valid driver in extracted netlist)

Lokpavan

8. Industry Mapping — EDA Tool Equivalence

8.1 Synopsys Custom Compiler / IC Validator

Dashboard Component	Synopsys ICV Equivalent
app.py upload + run button	ICV GUI job submission / ICVWB interface
python/main.py	icv binary / batch DRC run command
drc_engine.py	.drc rule deck (SVRF/ELT syntax execution)
lvs_engine.py	ICV-LVS netlist extraction + comparison engine
summary_report.txt	ICV sign-off .sum report
drc_violations.csv	ICV DRC results database (ICVWB violation viewer)
tcl/mock_drc_lvs.tcl	ICV TCL run script / SVRF batch control
[SIGNOFF] stdout token	ICV exit code + run summary file

8.2 Cadence Virtuoso / Pegasus

Dashboard Component	Cadence Equivalent
app.py	Virtuoso DRD / Constraint Manager UI
DRC rule structs	Pegasus .rs rule file (SKILL API)
LVS comparator	LVS hcell / Spectre netlist comparison
Violation tables (pandas)	Pegasus violation viewer / RVE database
report CSVs	Pegasus violation export to spreadsheet
JSON layout schema	OpenAccess database layer geometry

9. Deployment Guide

9.1 Local Setup

- Prerequisite: Python 3.11+ and pip installed.
- Clone or unzip the project folder: drc_lvs_dashboard/
- Install dependencies: pip install -r requirements.txt
- Launch the dashboard: streamlit run app.py
- Browser opens at: http://localhost:8501
- Upload sample_layout.json (included) to verify the installation.

9.2 GitHub Push

- git init && git add . && git commit -m 'Initial commit: DRC/LVS Dashboard'
- Create a public repository at github.com/new
- git remote add origin https://github.com/USERNAME/drc-lvs-dashboard.git
- git branch -M main && git push -u origin main

9.3 Streamlit Cloud Deployment (Free)

- Go to share.streamlit.io and sign in with GitHub.
- Click 'New app' and select the repository.
- Set Main file path to app.py and click Deploy.
- Live URL: https://USERNAME-drc-lvs-dashboard-app-XXXXX.streamlit.app
- Note: tclsh is unavailable on Streamlit Cloud — the TCL phase is gracefully skipped. DRC/LVS results are unaffected.

9.4 Render.com Deployment

- Create a Web Service and connect your GitHub repository.
- Build command: pip install -r requirements.txt
- Start command: streamlit run app.py --server.port $PORT --server.address 0.0.0.0
- Set runtime to Python 3.11 and deploy.

Lokpravan P

10. Apprenticeship JD Alignment — Synopsys

JD Requirement	Project Evidence
Python scripting & automation	4 Python modules, subprocess orchestration, argparse CLI, structured logging
Web application development	Streamlit dashboard with custom CSS, file upload, interactive tables
EDA tool integration	TCL invocation, ICV-style reporting, Synopsys/Calibre terminology throughout
Data handling & analysis	pandas DataFrame loading, Styler API, CSV generation, violation parsing
Cross-platform compatibility	Tested on Windows & Linux; cloud-safe (no tclsh dependency at runtime)
Version-controlled deliverable	Clean repo structure, .gitignore, README, requirements.txt
Structured reporting	3 report types: TXT summary, DRC CSV, LVS CSV — all downloadable
Understanding of IC design flows	Realistic rule names (M1.W.1, DIFF.S), LVS taxonomy, sign-off semantics
Software engineering best practices	Strict separation of concerns, modular design, error handling, no logic duplication

Lokpa

11. Technology Stack

Technology	Version	Role
Python	3.11+	Core engine language — DRC/LVS logic, orchestration, reporting
Streamlit	1.35+	Web UI framework — file upload, layout, widgets, download buttons
pandas	2.2+	CSV loading, DataFrame display, Styler API for violation colouring
TCL/tclsh	any	Mock EDA batch flow — phase logging, ICV-style run simulation
docx / npm	9.5+	Project report generation (this document)
JSON	stdlib	Layout input format — parsed natively with Python json module
Git / GitHub	any	Version control and cloud deployment source
Streamlit Cloud	free	Deployment target — zero-config, GitHub-integrated hosting

Lokpava

12. Conclusion

The DRC/LVS Verification Dashboard, developed by Lokpavan P, represents a complete, deployable, and professionally structured EDA automation project. It successfully bridges the gap between academic understanding and industry-grade tool implementation by replicating the verification flows used in Synopsys Custom Compiler and Cadence Virtuoso environments within a modern, accessible web interface.

The project demonstrates proficiency in all key skill areas relevant to a Software Engineering role at Synopsys: Python automation, EDA flow integration, TCL scripting, web development, structured reporting, and cloud deployment. The codebase is clean, modular, and production-ready, with clear documentation and a consistent separation of concerns between the UI layer, verification engine, and report generation subsystem.

This project is ready for immediate deployment to Streamlit Cloud via GitHub and can serve as both a working tool demonstration and a portfolio artefact for the Synopsys Software Engineering Apprenticeship application.

Developed by Lokpavan P

EDA Automation & Software Engineering Portfolio Project | 2025

Lokpavan P